RISC-V ISA Extension Systolic Array Acceleration for Quantized Inference

Spike Modification

Randy Revaldo Pratama
阮金鴻
115165425

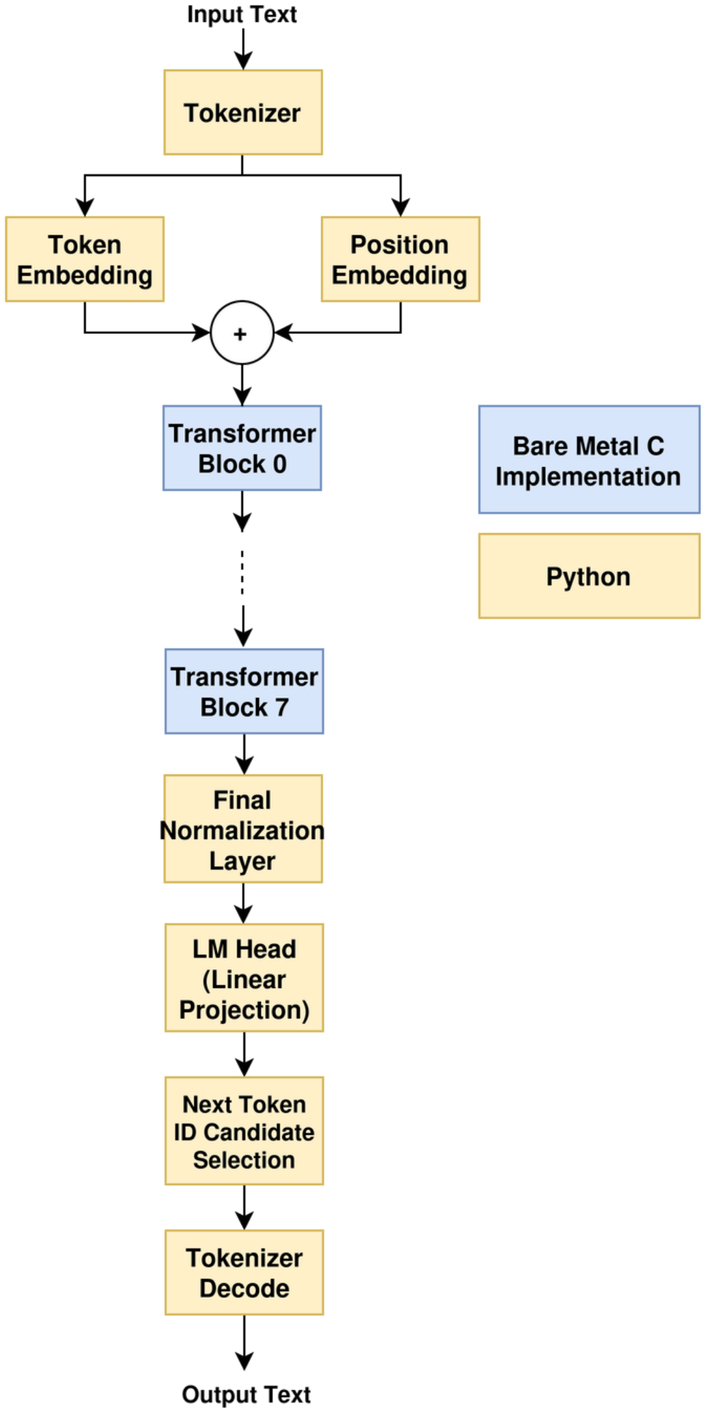
Outline

1. Model
2. Bottleneck Analysis
3. ISA Extension and Architecture
4. Result

Model

This model is a TinyStories-3M Language Model, with architecture Based on GPT-NEO

Parameter	Configuration
Input Tokens	32
Max Generated Tokens	20
Transformer Block	8
Quantization	W8A8

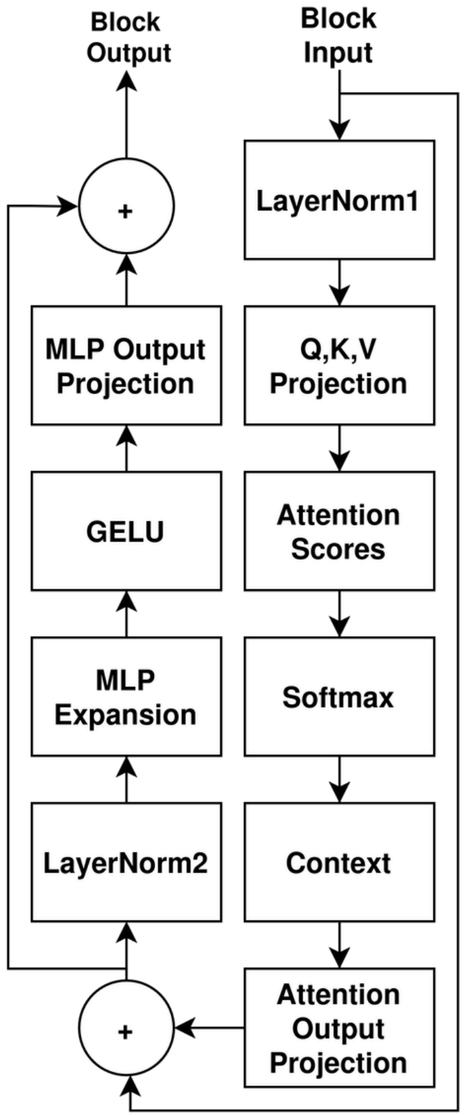


TinyStories explores how small a language model can be while still generating coherent English, by training on simple children's stories generated by GPT-3.5 and GPT-4.

Eldan, R., & Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv:2305.07759.

Bottleneck Analysis

Inside Transformer Block



Requantization To INT8 is done after:

- LayerNorm1
- Q,K,V Projection
- Context
- Attention Output Projection
- LayerNorm2
- MLP Expansion
- GELU
- MLP Output Projection
- Final Addition

Total Retired Instruction Across 8 Transformer Blocks

Region	Retired Instruction
LayerNorm1	2,117,309
QKV Projection	102,736,208
Attention Scores → Softmax → Context	23,437,008
Attention Output Projection	35,132,301
LayerNorm2	2,117,329
MLP Expansion → GELU	159,391,281
MLP Output Projection → Output	136,156,197
Total	461,087,565

Bottleneck Analysis

Transformer Block 0 Retired Instruction Analysis

Region	mul + mulw	add + addw	Other Instructions	Total
LayerNorm1	28,672	30,158	197,610	256,440
QKV Projection	1,585,152	1,622,409	9,634,435	12,841,996
Attention Scores → Softmax → Context	283,143	485,939	2,153,481	2,922,563
Attention Output Projection	536,576	553,093	3,277,048	4,366,717
LayerNorm2	28,672	30,017	197,690	256,379
MLP Expansion → GELU	2,359,296	2,490,536	14,746,108	19,595,940
MLP Output Projection → Output	2,105,344	2,125,957	12,710,136	16,941,437
Total	6,926,855	7,338,109	42,916,508	57,181,472

*Counted with the help of ChatGPT

ISA Extension and Architecture

XTILE Instruction

Systolic array based instruction set for matrix multiplication accelerator

- **Instruction Interface**

Instruction	Input values from CPU registers	Behavior
`xtile.cfg rs1`	One packed 64-bit configuration value	Validate and retain the configuration
`xtile.set_ab rs1, rs2`	A and BT base addresses	Retain both operand addresses
`xtile.set_c rs1`	C base address	Retain the output address
`xtile.set_zp rs1, rs2`	A and B zero points	Retain the centering parameters
`xtile.run`	None	Fetch, compute, and store the complete configured matrix operation

- **Configuration Register**

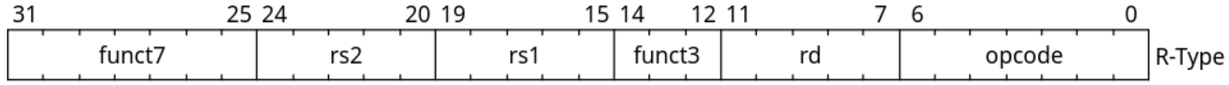
Bits	Field	Width	Description
`[63:38]`	RESERVED	26 bits	Must be zero
`[37:28]`	K	10 bits	Reduction length, stored directly
`[27:18]`	N	10 bits	Number of output columns, stored directly
`[17:8]`	M	10 bits	Number of output rows, stored directly
`[7:5]`	TILE_COLUMN	3 bits	Active tile columns minus one; encodes 1–8 columns
`[4:2]`	TILE_ROWS	3 bits	Active tile rows minus one; encodes 1–8 rows
`[1]`	B_TYPE	1 bit	`0`: signed INT8; `1`: signed INT32
`[0]`	A_TYPE	1 bit	`0`: signed INT8; `1`: signed INT32

ISA Extension and Architecture

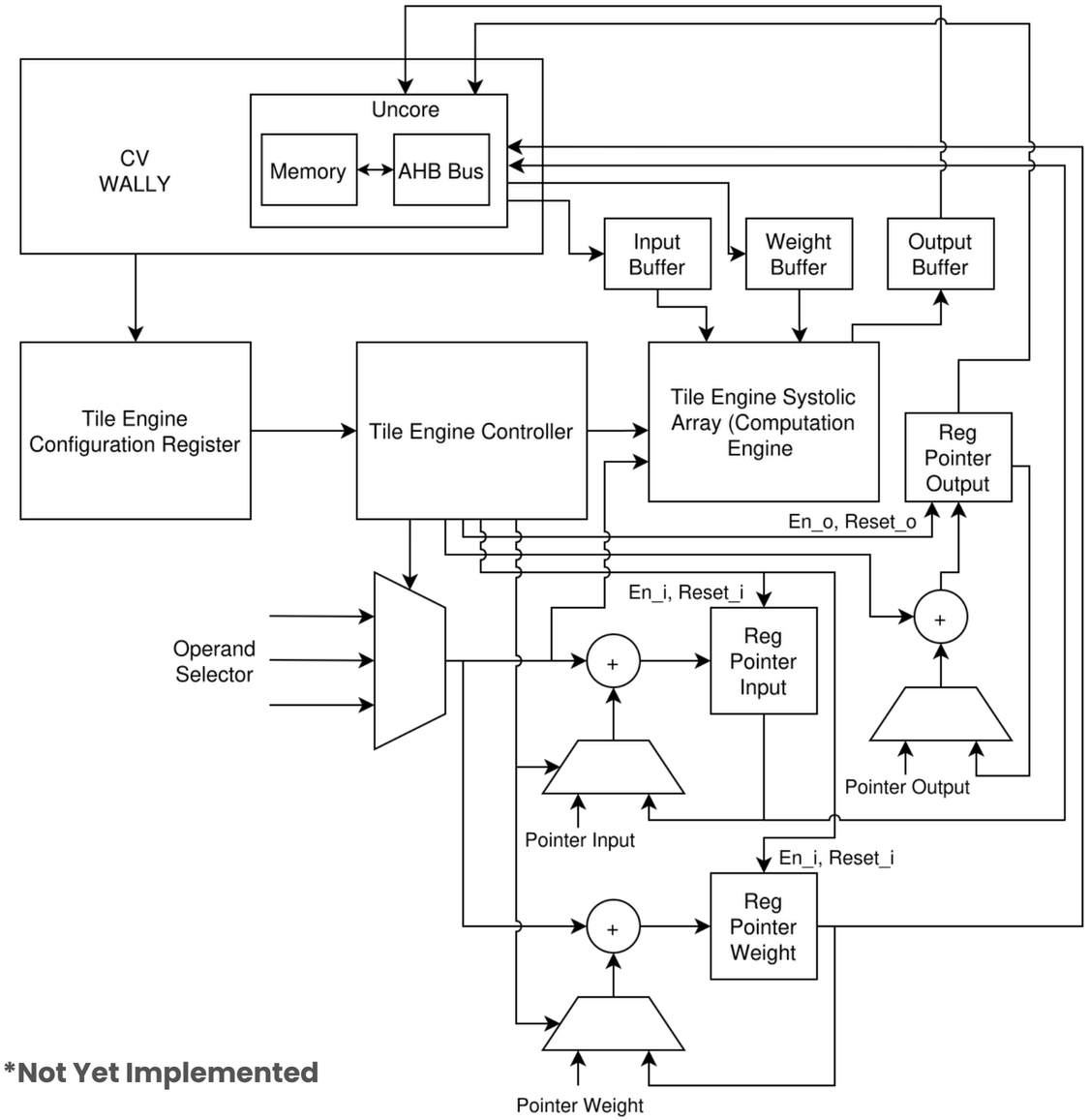
XTILE Instruction

- **Instruction Encoding**
  It uses custom-0 opcode space provided by RISC-V

Field	Bits	Value or role
funct7	`[31:25]`	Fixed zero
rs2	`[24:20]`	Second source register where used; otherwise x0
rs1	`[19:15]`	First source register where used; otherwise x0
funct3	`[14:12]`	Selects the XTILE operation
rd	`[11:7]`	Fixed x0; no CPU-register result
Opcode	`[6:0]`	`0001011`, hexadecimal `0x0b`



- **Overall Architecture**



*Not Yet Implemented

ISA Extension and Architecture

XREQUANT Instruction

Custom instruction set for the requantization

- **Instruction Interface**

Instruction	CPU-register inputs	Behavior
`xrequant.cfg rs1`	Packed 64-bit configuration	Validate and retain MODE, SHIFT, and COUNT
`xrequant.set_io rs1, rs2`	Source and destination base addresses	Retain both full RV64 addresses
`xrequant.set_mul rs1`	Signed INT64 multiplier	Retain the multiplier
`xrequant.set_zp rs1, rs2`	Input and output zero points	Retain signed INT32 zero points
`xrequant.run`	None	Convert and store the complete configured buffer

- **Configuration Register**

Bits	Field	Width	Meaning
`[63:41]`	RESERVED	23	Must be zero
`[40:9]`	COUNT	32	Number of elements, stored directly
`[8:3]`	SHIFT	6	Mode-dependent right shift
`[2:0]`	MODE	3	Conversion mode

- **Mode**

Mode	Source type	Destination type
0	INT64	INT32
1	INT64	INT8
2	INT32	INT8
3	INT8	INT32
4	INT32	INT8

***Two-stage round-clamp**

- **Mode 0 – 3** ***depends on each operation**

$$q_{\text{out}} = \text{Clamp}\left(\text{RoundShift}((q_{\text{in}} - z_{\text{in}}) \times M,\ s) + z_{\text{out}}\right)$$

- **Mode 4**

$$t = \text{RoundShift}(x \times M,\ 31)$$
$$y = \text{Clamp}_{[-128,127]}\left(\text{RoundShift}(t,\ s) + z\right)$$

***Two-stage rounding was used in the bare-metal implementation to match the LiteRT reference model's INT8 GELU output**

Result

Retired Instruction Reduction

*It does not mean the reduction in cycle, only reduction of the number of instructions

Region	Original Instruction	Custom Instruction	% Reduction
LayerNorm1	2,117,309	1,232,756	41.7772%
QKV Projection	102,736,208	1,636	99.9984%
Attention Scores → Softmax → Context	23,437,008	23,011,073	1.8174%
Attention Output Projection	35,132,301	592,363	98.3139%
LayerNorm2	2,117,329	1,232,771	41.7771%
MLP Expansion → GELU	159,391,281	9,704,984	93.9112%
MLP Output Projection → Output	136,156,197	527,024	99.6129%
Total	461,087,565	36,302,539	92.1268%

*Need adjustment for the matrix format to match the XTILE input format, so it mostly remains in scalar

Result

Cycle Reduction Estimation

Kernel / scope	Scalar cycles	Estimated array compute cycles	Scalar / array-compute ratio
Q projection	4,254,689	9,216	461.66×
MLP expansion	17,272,290	36,864	468.54×
MLP output	17,147,067	33,792	507.43×
One block — extrapolated	51,438,113	107,520	478.41×
Eight blocks — extrapolated	411,504,904	860,160	478.41×

*Calculation helped by ChatGPT

- **Only account for the systolic array with 8x8 Tiles**
- **Assume each PE has two multiplier stages followed by one accumulator stage (3 cycles)**

$$T_{tile} = K + (R - 1) + (C - 1) + (L - 1)$$

- **K = number of products accumulated for each output**
- **R = Row**
- **C = Column**
- **L = PE Cycles**

Thank you